Lecture 10 — How LLMs Work Conceptually



AI-Assisted Software Development and Intelligent Applications

Note: Welcome to Unit 2 — Smart Applications.

Today's Agenda

1. Tokenization: how text becomes numbers
2. Vector spaces and attention, conceptually
3. Temperature and sampling — why the same prompt gives different answers
4. **Activity** — Medical Knowledge Base: explore the starter app, then implement chunking

Pre-Class Checklist

Before we start — did you complete the reading?

- **The Illustrated Transformer** (Jay Alammar) — tokenization, attention weights, next-token prediction
- **Intro to Large Language Models** (Karpathy, first 30 min)

Today builds directly on both — we'll revisit the same diagrams.

Welcome to Unit 2 — Smart Applications

Unit 1: AI as a tool across the dev lifecycle. **Unit 2:** *building* applications where AI is a core component.

Two in-class arcs, each a parallel practice app (not your graded projects):

- **Lectures 10–14** — Medical Knowledge Base (today → real RAG by Lecture 13)
- **Lectures 15–19** — Automated Customer Support Agent (Claude SDK, tool use)

"Autocomplete on Steroids"

At its core, an LLM does one thing, repeatedly:

Given the text so far, predict the single most likely next token.

Chat, code generation, summarization — all of it is next-token prediction, fed back into the model as new context.

Useful mental model — but it breaks down when you expect *understanding* rather than *pattern continuation*.

Tokenization: Text → Numbers

Models don't read characters or words — they read **tokens**, chunks of text mapped to integer IDs by a tokenizer trained on a large corpus.

"What are the symptoms of hyperglycemia?"

What
3838
are
525
the
279
symptoms
13495

Why Token Counts Matter

- **Context windows** are measured in tokens, not characters — a model with a 128K context window fits ~128K tokens, and that budget includes your prompt *and* the response.
- **API cost** is usually billed per token, input and output separately.
- **"~4 characters per token" is a rough heuristic, not a rule.** It holds loosely for plain English prose — it breaks down for code, medical/legal jargon, and non-English text, all of which tokenize less efficiently.

Token Counts Aren't Universal

Each model family trains its own tokenizer on its own vocabulary — the **same text** produces a **different** token count on a different model. "Abetalipoproteinemia is inherited in an autosomal recessive manner."

Tokenizer	Token count
Qwen2.5 (today's tokenizer)	15
GPT-2	17

A token count from one model **does not transfer** to another — always count with the tokenizer for the model you're actually using.

Vector Spaces

Underneath tokenization, a model represents each token as a **vector** — a list of numbers, a point in high-dimensional space.

- Semantically similar concepts cluster near each other in that space
- `"diabetes"` sits closer to `"insulin"` than to `"aneurysm"`
- This is the same geometric idea that will power **embeddings** and vector search starting next lecture — today is the token-level version of that idea

Vector Spaces, Visualized



Illustrative sketch — real embeddings use hundreds of dimensions, not three.

Attention: Conceptually

When predicting the next token, the model doesn't weigh every prior token equally — it learns to **attend** more strongly to the tokens most relevant to what comes next.

```
"The patient with diabetes was prescribed ___"
```

To predict the blank, attention weights concentrate on `diabetes` and `prescribed` far more than on `The` or `was` — no heavy math required to get the intuition.

From Token Vectors to a Sentence Vector

"What are the symptoms of diabetes?"

What

are

the

symptoms

of

diabetes

?

each token already has its own vector (previous slides)

↓ attention lets every token's vector absorb context from every other

One Point, Same Space



The pooled sentence vector lands right in the diabetes cluster — exactly what Lecture 13's vector database compares.

Temperature and Sampling

The model outputs a **probability distribution** over the entire vocabulary, not one guaranteed next token. **Temperature** controls how that distribution gets sampled:

Temperature	Behavior
0.0	Nearly deterministic — highest-probability token
~0.7	Balanced — some variety, still coherent
1.5+	High randomness — more likely to go off the rails

Temperature 0.0 Isn't Fully Deterministic

Even at temperature 0.0, the *same* prompt can produce slightly different output across runs on different hardware.

Why: parallelized floating-point matrix operations aren't perfectly associative — tiny rounding differences can flip which token is technically "highest probability" in a near-tie.

Takeaway: never rely on exact reproducibility, even at temperature 0.

Where Sampling Meets Hallucination

Probabilistic sampling is *why* hallucinations happen at all:

- The model is always predicting a plausible next token — not looking anything up
- A fluent, confident-sounding continuation can be entirely fabricated
- Higher temperature increases hallucination risk; temperature 0 reduces but does not eliminate it

This is the core reason Lecture 11 introduces **retrieval** — giving the model real source text to ground its answer in, instead of trusting it

Activity — Medical Knowledge Base

A **pre-built** full-stack starter app (React/TypeScript + FastAPI + MySQL, via Docker Compose) — plain keyword search, no AI yet. You'll grow this same app across Lectures 10–14 until it runs real local RAG.

Already loaded with **three data types**, chosen to mirror the heterogeneous resources Project 3 will require you to ingest:

- **Type A** — MedQuAD Q&A pairs (atomic)
- **Type B** — a MedlinePlus health-topic page with real section headers and markup

Activity — Two Parts

Part 1 — Explore (no code): bring up the Docker stack, search the app, and look at how the three data types come back from `/api/search` — notice this is plain substring matching with no understanding of meaning.

Part 2 — Tokenize (`tokenizer_lab/`, plain Python, no Docker): run `explore_tokens.py` to see how token counts differ across the three data types, then implement `chunk_text()` — a word-level sliding-window chunker — so all 6 pytest tests pass. This is today's only code change.

Why chunking now? Every RAG pipeline has to fit documents into a

Questions?



Next lecture: What is Retrieval-Augmented Generation (RAG)? — read the RAG paper (Lewis et al., Sections 1–2) and the ChromaDB Core Concepts docs before class